Cloud Native Data Engineering

SQL Reference

Ashish Pujari | apujari@uchicago.edu

Lecture Outline

1. SQL Introduction
2. Selecting Data
3. Aggregating Data
4. Sorting and Grouping Data
5. Combining Data
6. Nested Queries
7. Manipulating Data
8. Views
9. Transforming Data
10. Transactions, Security, and Stored Programs
11. Advanced Queries
12. Query Performance
13. Specialized SQL Features
14. SQL in Python



SQL Introduction

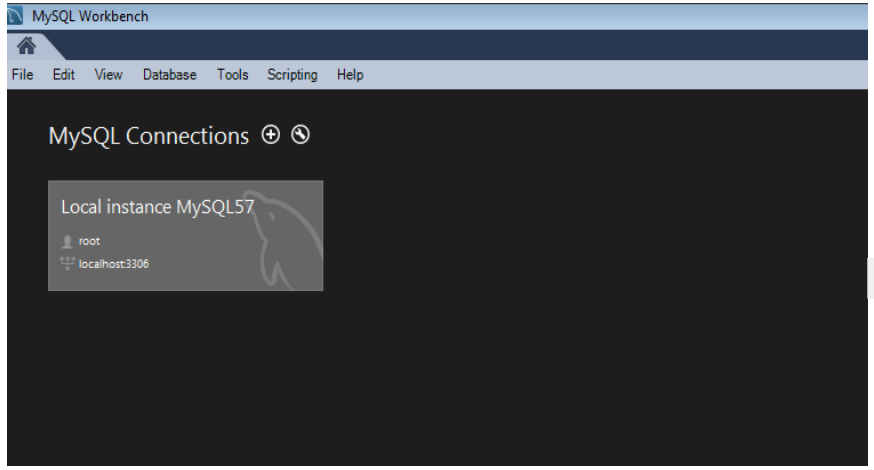
Module 1

Structured Query Language (SQL)

- The ANSI standard language for the definition and manipulation of relational databases
- SQL is declarative: describe WHAT you want, not HOW to retrieve it
- First version developed at IBM by Donald D. Chamberlin and Raymond F. Boyce [SQL]
- Based on Dr. E.F. Codd's paper, “A Relational Model of Data for Large Shared Data Banks”

SQL Session

- A SQL session is an occurrence of a user interacting with a relational database through the use of SQL commands
- When a user connects to a database, the SQL session is initialized. The CONNECT command is used to establish a database connection



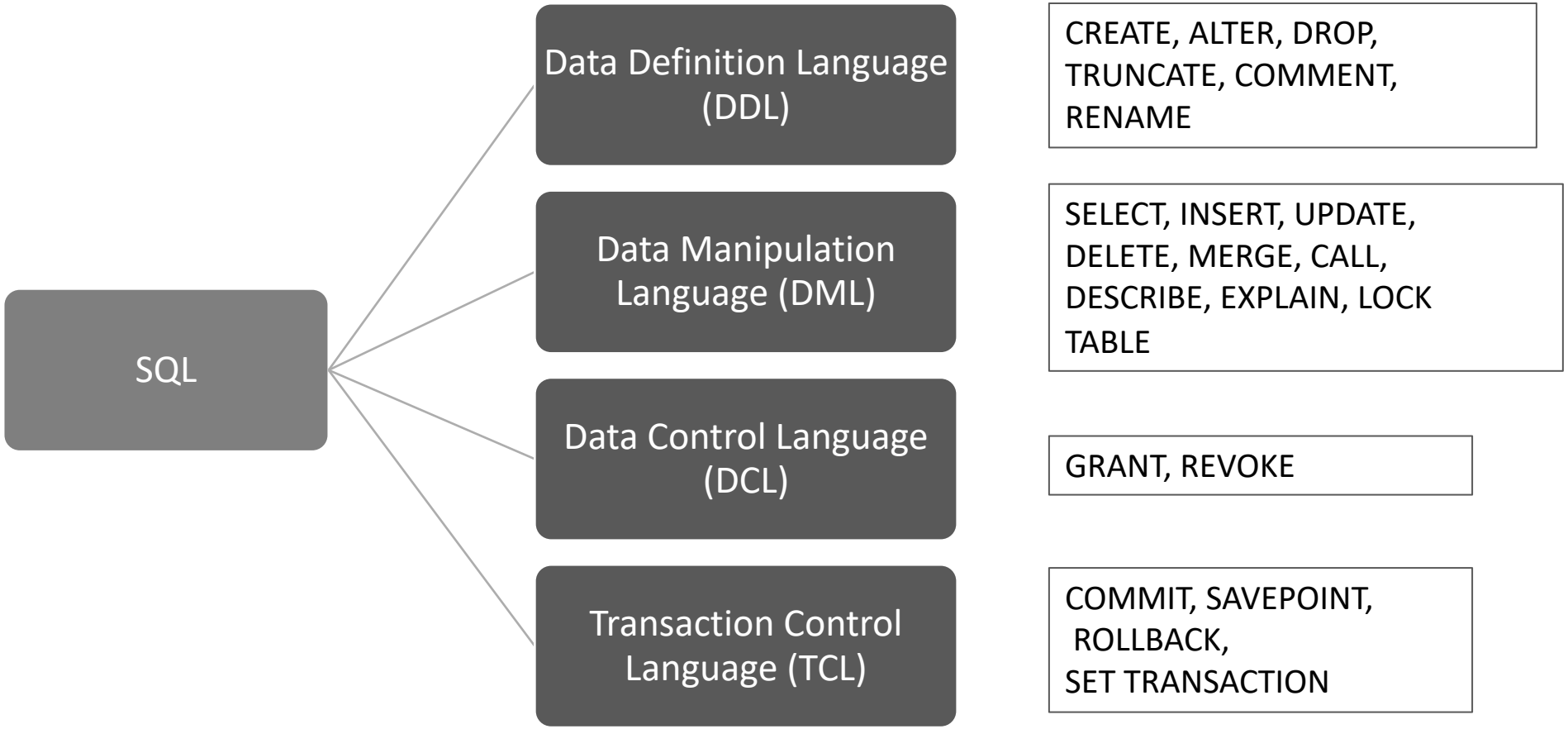
MySQL Workbench

SQL Connection



Database

SQL Command Types



DDL vs. DML

DDL — Data Definition Language

CREATE, ALTER, DROP, TRUNCATE

Defines and changes schema structure: tables, columns, constraints, indexes

Most engines auto-commit DDL immediately — hard or impossible to roll back

Lab: SQL-3.8-DDL.sql — create and alter tables against the course schema

DML — Data Manipulation Language

SELECT, INSERT, UPDATE, DELETE

Reads and writes rows within structures that already exist

Participates in transactions — safe to wrap in BEGIN … COMMIT / ROLLBACK

Everything in Module 1's query examples (CTEs, subqueries, joins) is DML

CRUD Operations Map to DML Statements

Operation	SQL statement
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

DDL - Data Definition Language

Command	Purpose
CREATE	Creates objects in the database
ALTER	Alters the structure of an object
DROP	Deletes objects from the database
TRUNCATE	Removes all rows from a table, keeping its structure
COMMENT	Adds comments to the data dictionary
RENAME	Renames an object

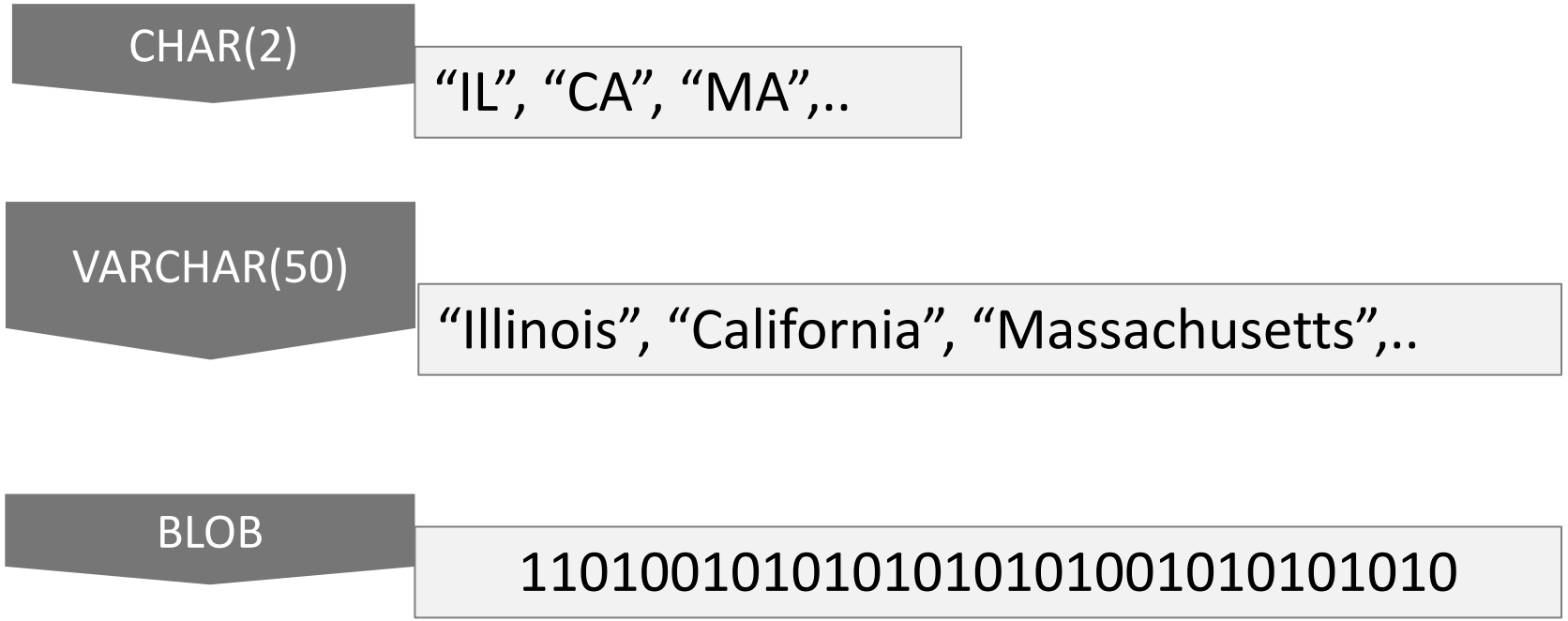
DML - Data Manipulation Language

Command	Purpose
SELECT	Retrieves data from a database
INSERT	Inserts data into a table
UPDATE	Updates existing data within a table
DELETE	Deletes rows from a table (all rows if there is no WHERE clause)
DESCRIBE	Shows information about a table's structure
EXPLAIN	Shows the access path to the data (query execution plan)

String Data Types

String Types	Description
CHAR	A fixed-length non-binary (character) string
VARCHAR	A variable-length non-binary string
BINARY	A fixed-length binary string
VARBINARY	A variable-length binary string
TINYBLOB	A very small BLOB (binary large object)
BLOB	A small BLOB
MEDIUMBLOB	A medium-sized BLOB
LONGBLOB	A large BLOB
TINYTEXT	A very small non-binary string

String Data Types - Examples



Numeric Data - Integer Types

Numeric Types	Description	Storage (Bytes)	Min – Max values signed, unsigned
TINYINT	A very small integer	1	[-128 to 127], [0 to 255]
SMALLINT	A small integer	2	[-32768 to -32767], [0 to 65535]
MEDIUMINT	A medium-sized integer	3	[-8388608 to 8388607], [0 to 16777215]
INT	A standard integer	4	[-2147483648 to 2147483647], [0 to 4294967295]
BIGINT	A large integer	8	[-9223372036854775808 to 9223372036854775807], [0 to 18446744073709551615]

Numeric Data – Fixed and Floating Point

Numeric Types	Description	Type
DECIMAL (M,D) or NUMERIC (M,D)	A fixed-point number	Exact
FLOAT (M,D)	A single-precision floating point number	Approximate
DOUBLE (M,D)	A double-precision floating point number	Approximate

(M,D) means than values can be stored with up to M digits in total, of which D digits may be after the decimal point.

e.g. DECIMAL(5,2) -999.99 to 999.99.

Date and Time Data Types

Date Types	Description
DATE	A date value in ‘CCYY-MM-DD’ format
TIME	A time value in ‘hh:mm:ss’ format
DATETIME	A date and time value in ‘CCYY-MM-DD hh:mm:ss’ format
TIMESTAMP	A timestamp value in ‘CCYY-MM-DD hh:mm:ss’ format
YEAR	A year value in CCYY or YY format



Selecting Data

Module 2

Select Clause

- The SELECT clause specifies the columns from which data values are to be retrieved by the query.
- Data retrieval is limited to the columns specified
- To display all the columns of a table, use the asterisk wildcard character (*)

```
SELECT column_name,column_name
FROM table_name;
```

Where Clause

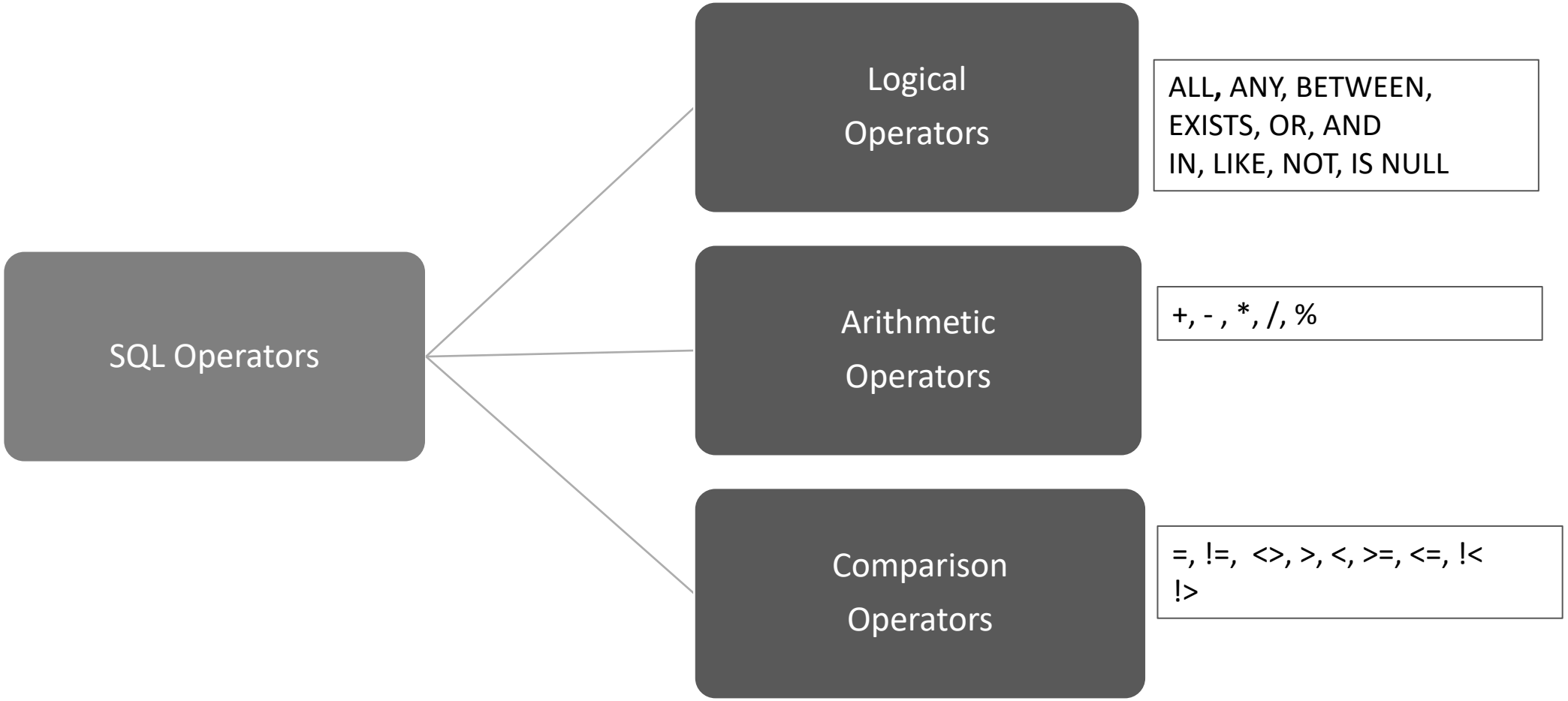
- The WHERE clause is used to filter records
- All rows for which the predicate in the WHERE clause is True are returned by the SQL DML statement or query criterion

```
SELECT column_name,column_name
FROM table_name
WHERE column_name operator value;
```

SQL Operator

- An operator performs on separate data items and returns a result. The data items are called operands or arguments
- Reserved word or character used primarily in WHERE clause to perform comparisons and arithmetic operations

Operator Types



Logical Operators

LOGICAL OPERATORS	DESCRIPTION
ALL	Compare a value to all values in another value set.
ANY	Compare a value to any applicable value in the list according to the condition.
BETWEEN	Search for values that are within a set of values, given the minimum value and the maximum value.
EXISTS	Search for the presence of a row in a specified table that meets certain criteria.
OR	At least one of the conditions must be true
AND	All the specified conditions must be true
IN	Compare a value to a list of literal values that have been specified.
LIKE	Compare a value to similar values using wildcard operators.
NOT	Reverses the meaning of the logical operator. This is a negate operator.
IS NULL	The NULL operator is used to compare a value with a NULL value.

Arithmetic Operators

ARITHMETIC OPERATORS	DESCRIPTION
+ (ADD)	Addition
- (SUBTRACT)	Subtraction
* (Multiply)	Multiplication
/ (Divide)	Division
% (Modulo)	Integer remainder of a division

Comparison Operators

COMPARISON OPERATORS	DESCRIPTION
=	Checks if the values of two operands are equal or not, if yes then condition becomes true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.
<>	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.
!<	Checks if the value of left operand is not less than the value of right operand, if yes then condition becomes true.
!>	Checks if the value of left operand is not greater than the value of right operand, if yes then condition becomes true.

Operator Precedence

```
INTERVAL
BINARY, COLLATE
!
- (unary minus), ~ (unary bit inversion)
^
*, /, DIV, %, MOD
-, +
<<, >>
&
|
= (comparison), <=>, >=, >, <=, <, <>, !=, IS, LIKE, REGEXP, IN
BETWEEN, CASE, WHEN, THEN, ELSE
NOT
AND, &&
XOR
OR, ||
= (assignment), :=
```



Highest

Lowest

Select Case

- SELECT CASE is used for equality tests of one expression against multiple values

```
CASE expression
  WHEN test THEN result
  …
  ELSE other Result
END;
```

Select Distinct

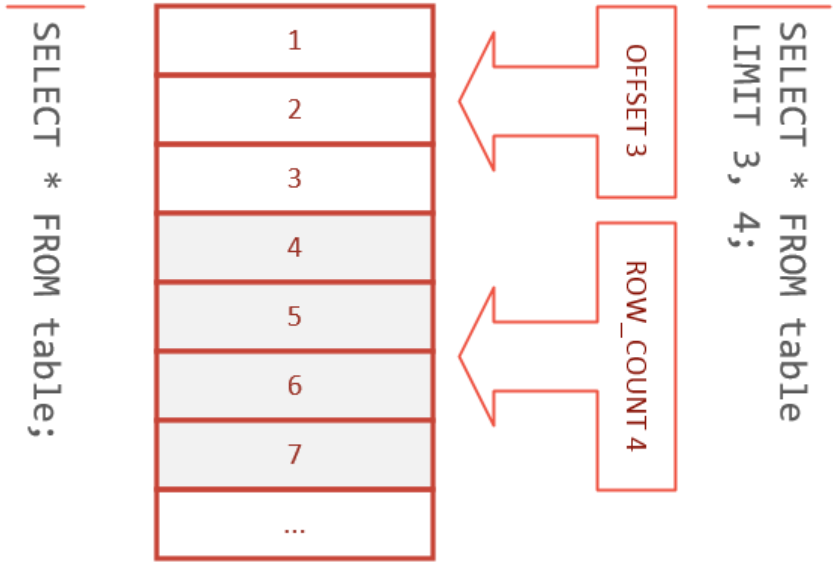
- SELECT DISTINCT statement is used to return only distinct (different) values.

```
SELECT DISTINCT column_name,column_name
FROM table_name;
```

Limit

- LIMIT clause is used to specify the number of records to return
- Useful on large tables with thousands of records; avoids performance issues

```
SELECT column_name(s)
FROM table_name
LIMIT number OFFSET offset
```



https://www.sqltutorial.org/sql-limit/

Practice Topics: Selecting Data

- SELECT and WHERE clauses
- Arithmetic, Logical and Comparison Operators
- Operators used singularly and in combinations
- Conditional Statements
- Basic use of DISTINCT and LIMIT



Aggregating Data

Module 3

Aggregate Function

- Performs a calculation on a set of numerical values and returns a single value
- Ignores NULL values when it performs calculation
- Required by GROUP BY clause

```
SELECT AGGREGATE FUNCTION (column_name)
FROM table_name;
```

Count (expr) Function

- Returns a count of the number of non-NULL values of expression in the rows retrieved by a SELECT statement
- The result is a BIGINT value

```
SELECT COUNT(expression)
FROM table_name;
```

Count (*) Function

- Returns a count of the number of rows retrieved, whether or not they contain NULL values
- Sometimes optimized when SELECT retrieves from one table, and there is no WHERE clause

```
SELECT COUNT(*)
FROM table_name;
```

Aggregate Functions

AGGREGATE FUNCTIONS	DESCRIPTION
AVG()	Return the average value of the argument
COUNT()	Return a count of the number of rows returned
COUNT(DISTINCT)	Return the count of a number of different values
MAX()	Return the maximum value
MIN()	Return the minimum value
STD()	Return the population standard deviation
STDDEV()	Return the population standard deviation
STDDEV_POP()	Return the population standard deviation
STDDEV_SAMP()	Return the sample standard deviation
SUM()	Return the sum
VARIANCE()	Return the population standard variance

Practice Topics: Aggregating Data

- COUNT(*) and COUNT(column) functions
- COUNT with DISTINCT and WHERE clause
- Aggregate functions - SUM, MIN, MAX, AVG
- Aggregate functions - Statistical
- Aggregate functions with conditional statements



Sorting and Grouping Data

Module 4

Order By Clause

- Sort data in ascending or descending order, based on one or more columns
- Default sort is ascending

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator value
ORDER BY column_name, column_name;
```

Group By Clause

- Used with the SELECT statement to arrange identical data into groups
- Selected columns must appear in the GROUP BY clause
- Follows the WHERE clause and precedes the ORDER BY clause

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator value
GROUP BY column_name, column_name
ORDER BY column_name, column_name;
```

Order By Vs Group By

- All non-aggregate columns selected must be listed in the GROUP BY clause
- GROUP BY clause generally not necessary unless using aggregate functions

Having Clause

- When used in conjunction with the GROUP BY clause in a SELECT statement, tells which groups to include in the output
- Must follow the GROUP BY clause and precede ORDER BY

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator value
GROUP BY column_name, column_name
HAVING conditions
ORDER BY column_name, column_name;
```

Practice Topics: Sorting and Grouping Data

- Sorting data using ORDER BY clause
- Aggregating and grouping data using GROUP BY clause
- Sorting and Grouping with conditional statements
- Listing top results using LIMIT

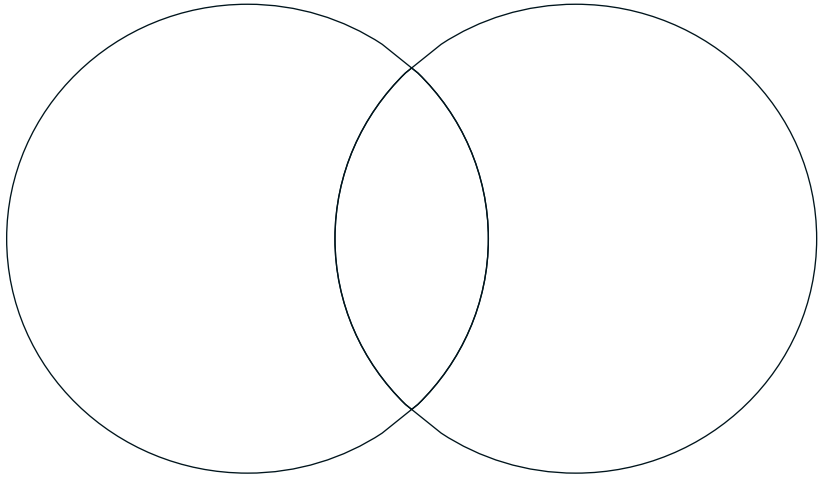


Combining Data

Module 5

SQL Joins

- SQL JOIN clause is used to combine rows from two or more tables, based on a common attribute
- Types of Joins
  - Inner Join
  - Outer Join (Left, Right, Full)
  - Cross Join

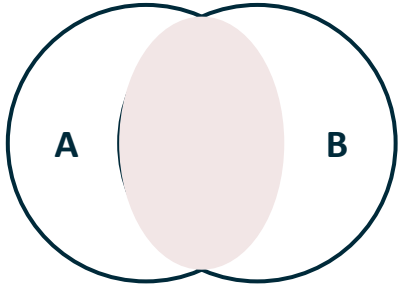


E.g. Relational tables Table 1 and Table 2 related by PK, FK

Join Types at a Glance

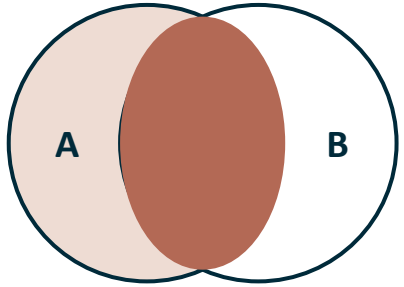
INNER JOIN

matching rows only



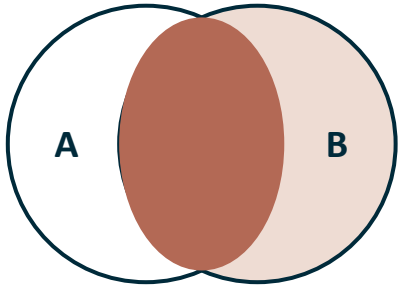
LEFT JOIN

all of A + matching B



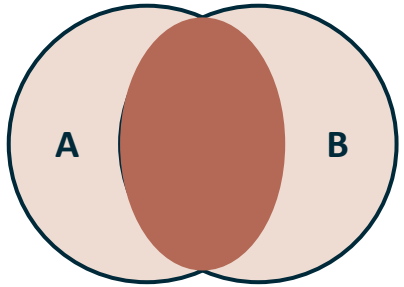
RIGHT JOIN

all of B + matching A



FULL OUTER JOIN

all rows from both tables

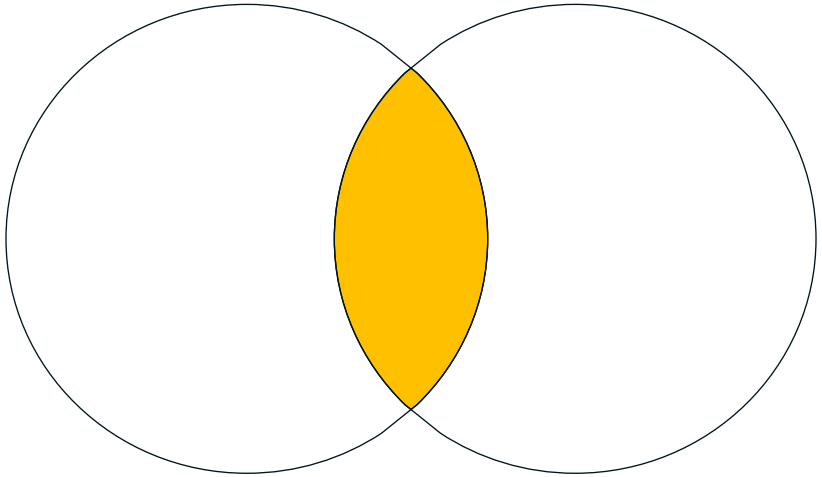


ROWS RETURNED BY THE JOIN

LEFT is most common for analytics — keeps all driving records

Inner (Simple) Join

- The INNER JOIN keyword selects all rows from both tables as long as there is a match between the columns in both tables.
- Most common type of join




```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name=table2.column_name;
```

Left Outer Join

- Returns all rows from the left table (Table1), with the matching rows in the right table (Table2).
- The result is NULL in the right side when there is no match.

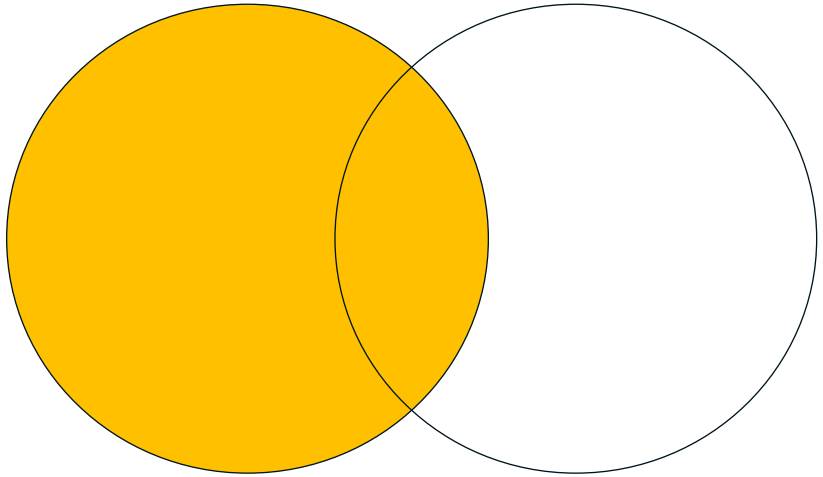


Table1 Table2

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name=table2.column_name;
```

Right Outer Join

- Returns all rows from the right table (Table2), with the matching rows in the left table (Table1).
- The result is NULL in the left side when there is no match.

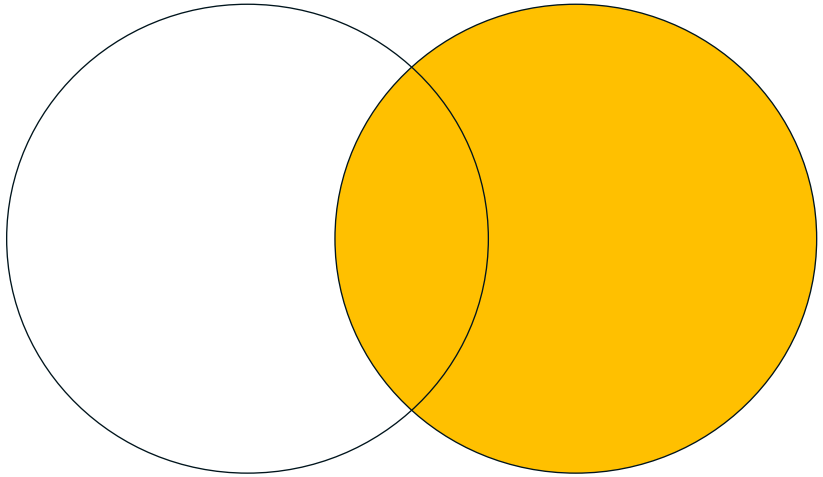


Table1 Table2

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name=table2.column_name;
```

Full Outer Join

- Returns all rows from the left table (Table1) and from the right table (Table2).
- Combines the result of both LEFT and RIGHT joins.

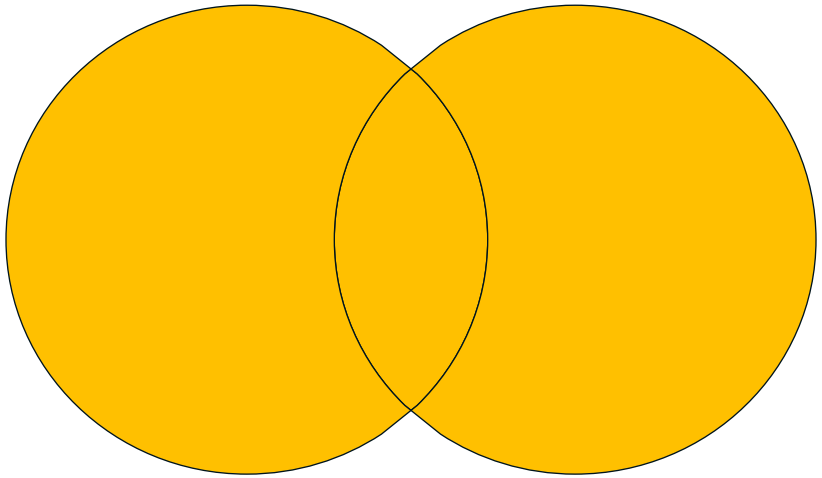


Table1 Table2

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name=table2.column_name;
```

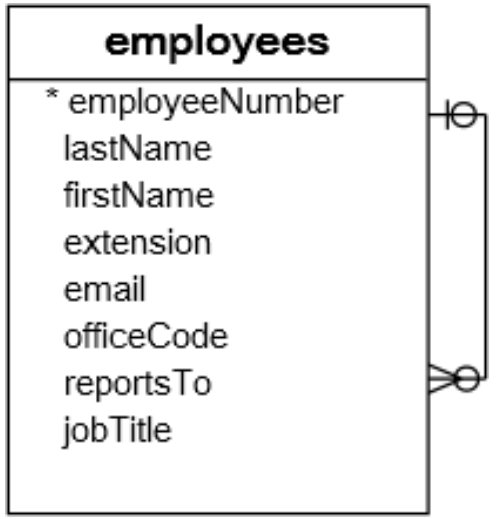
Cross (Cartesian) Join

- Cartesian product of the tables that involved in the join
- Returns (rows in Table1) × (rows in Table2)
- Could lead to performance issues due to large result set

```
SELECT column_name(s)
FROM table1 t1,
CROSS JOIN table2 t2
```

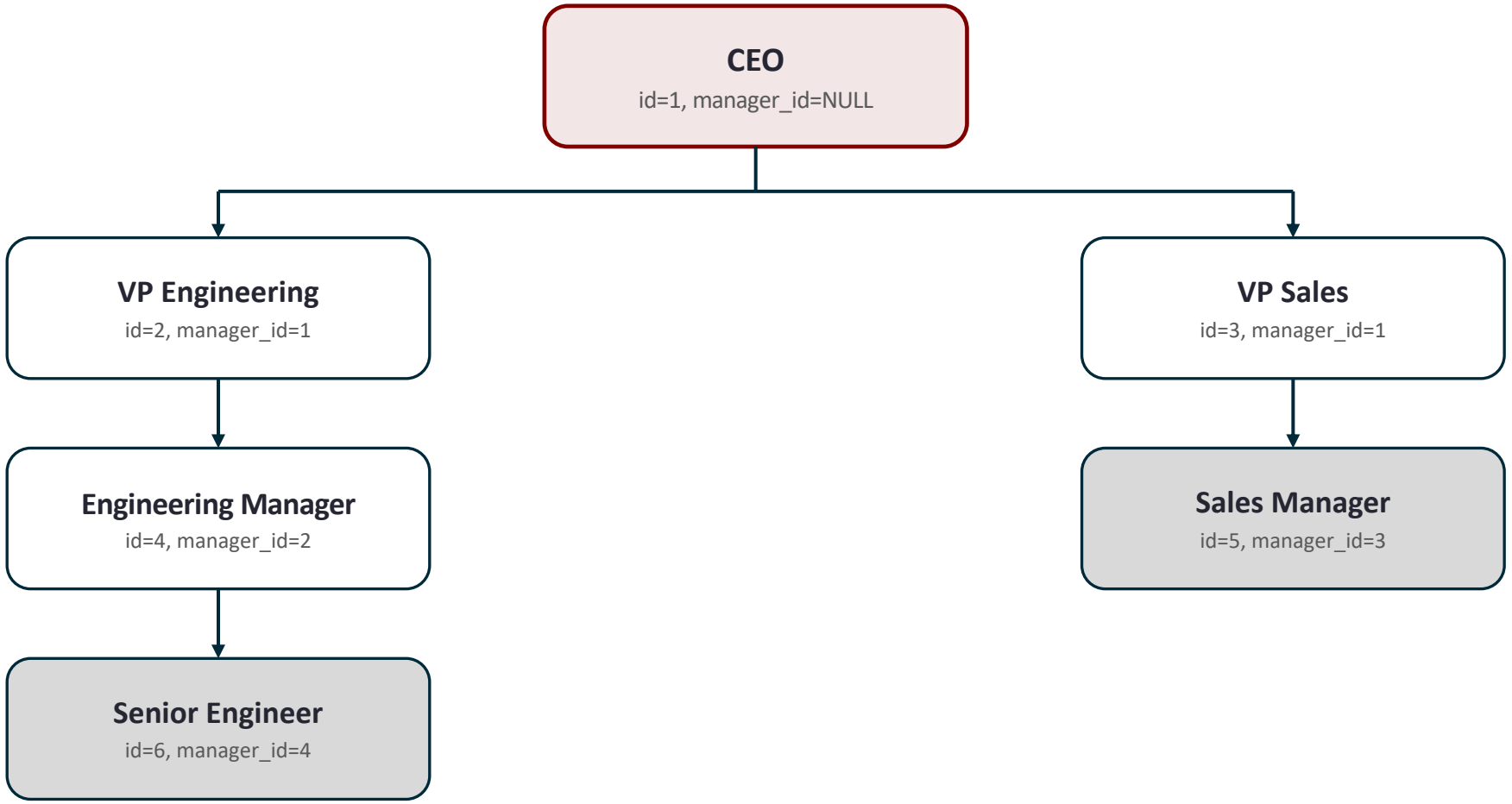
Self Join

- Used to join a table to itself, as if the table were two tables
- E.g. Employees table org structure report




```
SELECT column_name(s)
FROM table1 t1,
INNER JOIN table1 t2
ON t1.column1 = t2.column2
```

Self Join: Employee Hierarchy



SELF JOIN e AS employee JOIN e AS manager ON e.manager_id = manager.id

Union

- Used to combine two or more result sets from multiple tables into a single result set
- Number of columns in the corresponding select statements must be equal and of the same/convertible data type.
- UNION eliminates duplicate rows by default
- UNION ALL does not eliminate duplicate rows

```
SELECT column_name(s)
UNION [DISTINCT | ALL]
SELECT column_name(s)
```

Practice Topics: Combining Data

- Inner Join
- Outer Join
  - Left Join
  - Right Join

- Natural Join
- Self Join
- Union



Nested Queries

Module 6

Nested Query

- Query that is nested inside another query
- Also called an Inner query or Subquery while the query that contains it is called an Outer Query

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator value OPERATOR
        (SELECT column_name
        FROM table_name
        WHERE column_name operator value)
```

Outer Query

Inner Query

Nested Query Rules

- Nested query must always appear within parentheses
- Possible to have as many level of nesting as needed
- Can be nested inside SELECT, INSERT, DELETE, or UPDATE
- May occur in SELECT, FROM, WHERE and HAVING

Advantages of Nested Queries

- Allow the use of results of another query in the outer query
- Complex queries can be broken down into a series of logical steps for easy understanding and code maintenance
- Can replace complex joins and unions in some cases

Practice Topics: Nested Queries

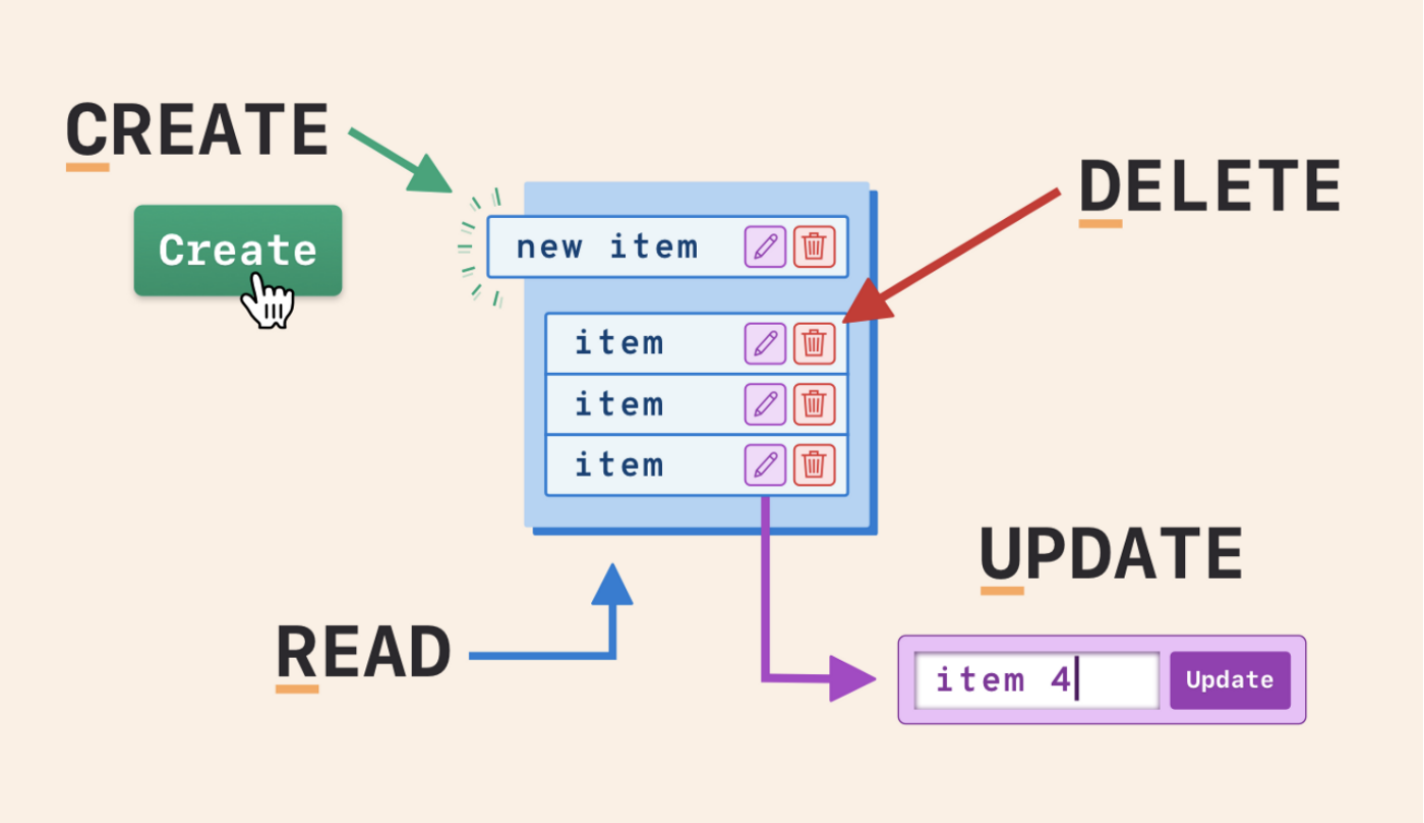
- Nested queries with IN and NOT IN operators
- Nested queries with EXISTS and NOT EXISTS
- Nested query in FROM clause
- Nested query in SELECT statement



Manipulating Data

Module 7

CRUD at a Glance



Create/Drop Database

- Create - Used to create a new database schema

```
CREATE DATABASE database_name;
```

- Drop - Used to drop or delete an existing database

```
DROP DATABASE database_name;
```

Create Table

- Used to create a table in a database
  - The column parameters specify the names of the columns of the table.
  - The datatype parameter specifies the type of data the column can hold (e.g. varchar, integer, date, etc.).

```
CREATE TABLE table_name (
   column1 datatype,
   column2 datatype,
  ....
);
```

Insert Statement

- Used to insert new records into a table in a database
  - Specify both the column names and the values to be inserted

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

  - If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query.

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Update Statement

- Used to update existing records in a table
  - The WHERE clause specifies which record(s) that should be updated.
  - If the WHERE clause is omitted, all records in the table will be updated.

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Delete Statement

- Used to delete existing records in a table
  - The WHERE clause specifies which record(s) should be deleted
  - If the WHERE clause is omitted, all records in the table are deleted

```
DELETE FROM table_name
```

Practice Topics: Manipulating Data

- Create database
- Create tables
- Insert data
- Update data
- Delete the database

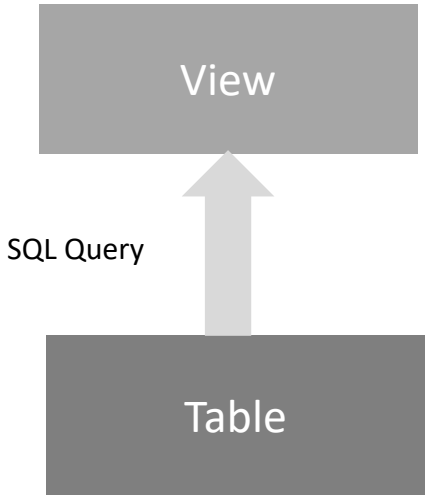


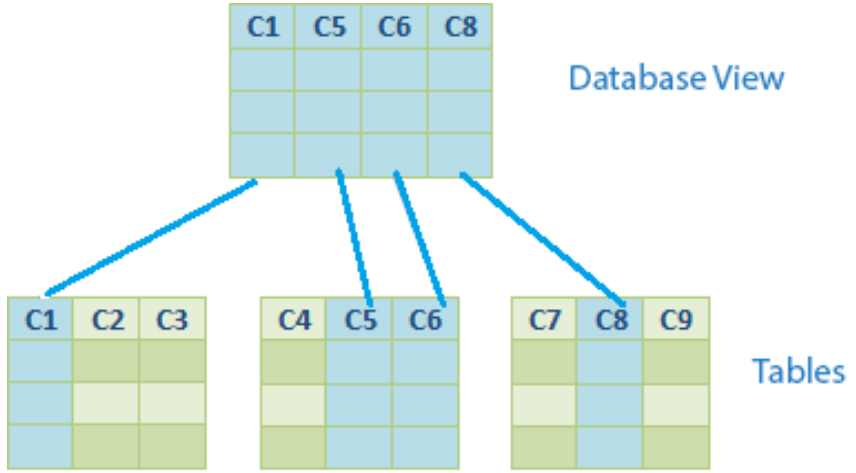
Views

Module 8

View

- A virtual or logical table that doesn't require physical storage
- Also called a named query or a stored query
- Data can be manipulated in a view, with some restrictions





Advantages of Views

Purpose	How views help
Simplify data access	Hide the complexity of multi-table joins and SELECT statements; enable backward compatibility
Maintain data security	Restrict user access to particular columns or rows; control access for different types of users in large multi-user systems
Maintain summarized data	Computed columns: business logic, sums, averages, etc.

Create and Drop View

- View can be created from one or more tables
- View be deleted like other database objects
- View may also be created from another view

```
CREATE OR REPLACE VIEW view_name AS
      SELECT <statement>;
```

```
DROP VIEW view_name;
```

```
CREATE VIEW view1
AS SELECT * FROM  view2
```

Restrictions on Views

Creating a view	Updating through a view
The SELECT cannot contain a subquery in the FROM clause	Not allowed with aggregate functions, DISTINCT, GROUP BY, HAVING, or UNION
The SELECT cannot refer to system or user variables	Not allowed with a subquery in the SELECT list
A temporary view cannot be created	Not allowed with certain joins
A view cannot be associated with a trigger	Refer to the MySQL documentation for the full list
View rows and underlying table rows must have a 1:1 relationship	

Materialized Views

- Materialized view creates and stores the result table in advance, filled with data
- The scheme of this table is given by the SELECT clause of the view definition
- Useful when the query involves many joins of large tables, or any query that has long execution times

Practice Topics: Views

- Create views with SELECT statements
- Update data in underlying tables by updating views
- Enforce data integrity using CHECK OPTION
- Delete views



Transforming Data

Module 9

String Functions

String Functions	Description
LOWER() /UPPER()	Returns the argument in lowercase/uppercase
TRIM()	Removes leading and trailing spaces
LTRIM() /RTRIM()	Removes leading/trailing spaces
LPAD()/ RPAD()	Returns the string argument, left-padded/right-padded with the specified string
CONCAT()	Returns concatenated string
SUBSTRING(), SUBSTR()	Returns the substring
LOCATE(),POSITION()	Returns the position of the first occurrence of substring

Numeric Functions

Numeric Functions	Description
ABS()	Returns the absolute value of numeric expression
CEIL()	Returns the smallest integer value that is not less than passed numeric expression
FLOOR()	Returns the largest integer value that is not greater than passed numeric expression
LOG()	Returns the natural logarithm of the passed numeric expression
EXP()	Returns e (based of natural log) raised to the power of passed numeric expression
POW()	Returns the value of one expression raised to the power of another expression
ROUND()	Used to round an expression to a number of decimal points
SQRT()	Returns the non-negative square root of numeric expression

Date Functions

Date Functions	Description
NOW()	Returns the current date and time
CURDATE()	Returns the current date
DATE()	Extracts the date part of a date or date-time expression
DATEDIFF()	Subtracts two dates
DATE_ADD()	Adds two dates
SYSDATE()	Returns the time at which the function executes
DAYOFMONTH()	Returns the day of the month (1-31)
DAYOFWEEK()	Returns the weekday index of the argument
DAYOFYEAR()	Returns the day of the year (1-366)

Data Conversion - Implicit

- Converts a value of any type into a value that has a specified type
- Target type can be any one of the following types: BINARY, CHAR, DATE, DATETIME, TIME,DECIMAL, SIGNED, UNSIGNED.
- MySQL automatically converts some data types for comparison

```
mysql> SELECT 1+'1';
-> 2
```

Data Conversion - CAST, CONVERT

- Converts a value of any type into a value that has a specified type
- CONVERT() with USING converts data between different character sets

```
CAST(expression AS TYPE);

CONVERT(expression, type);

CONVERT(expr USING transcoding_name);
```

E.g.

```
CAST ( $210.27 AS VARCHAR(10) )
```

Practice Topics: Transforming Data

- String Functions
- Numeric Functions
- Date Functions
- Convert data types using CAST



Transactions, Security, and Stored Programs

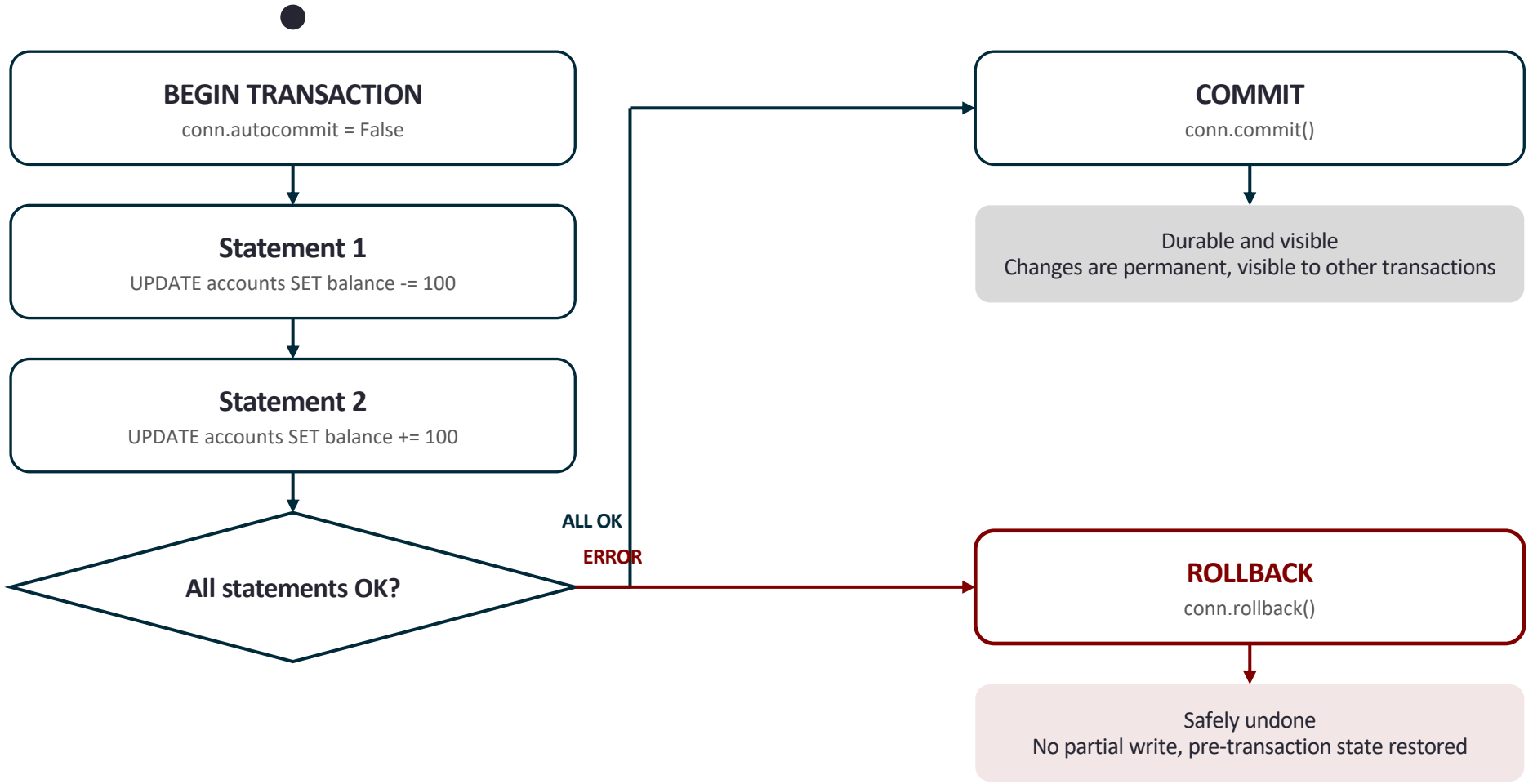
Module 10

TCL - Transaction Control

Command	Purpose
COMMIT	Saves the work done
SAVEPOINT	Identifies a point in a transaction to which you can later roll back
ROLLBACK	Restores the database to its state at the last commit
SET TRANSACTION	Changes transaction options such as the isolation level

TCL groups statements into logical transactions.

Transactions and Rollback



Python: wrap statements in try/except — commit() on success, rollback() on any exception

DCL – Data Control Statements

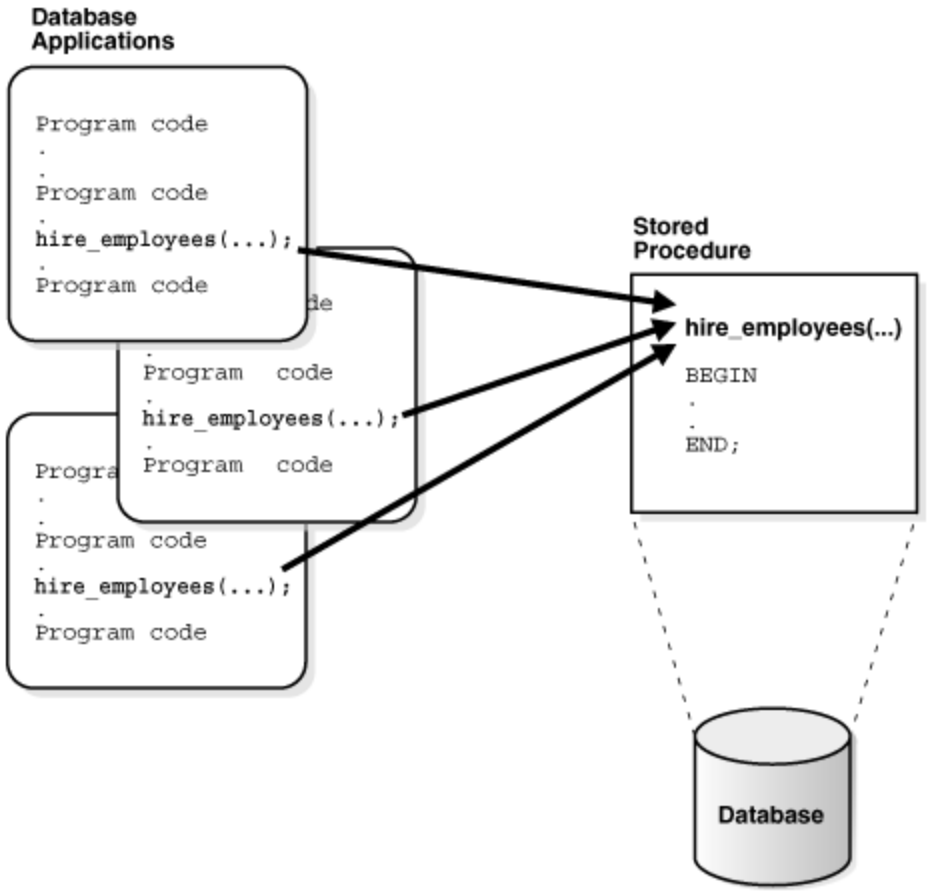
Command	Purpose
GRANT	Gives users access privileges to the database
REVOKE	Withdraws access privileges given with GRANT

```
GRANT SELECT, INSERT, UPDATE, DELETE ON table_name TO user_name;

REVOKE INSERT ON table_name TO user_name;
```

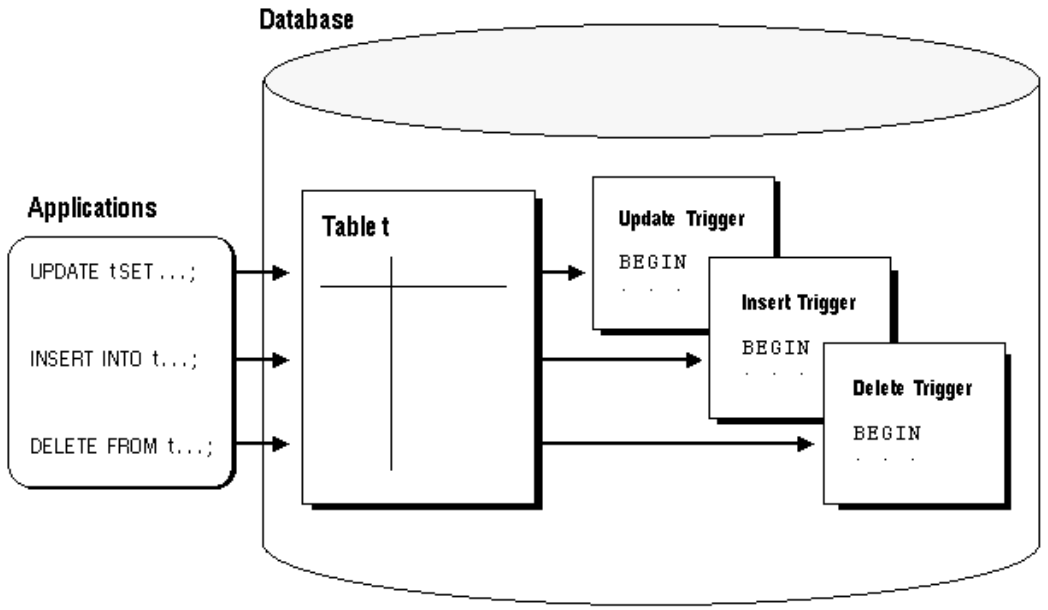
Stored Procedures

- A stored procedure is a named set of SQL statements stored in the database
- It can be shared by a number of programs



Triggers

- A trigger is a special type of stored procedure
- A trigger is called automatically when a data modification event is made against a table whereas a stored procedure must be called explicitly



Stored Procedures, Functions, and Triggers

Object	How it runs	Typical use
Stored procedure	Called explicitly with CALL; does not return a value directly; accepts parameters	Multi-step logic; fewer network round-trips
Stored function	Returns a value; usable in SELECT and WHERE clauses	Reusable calculations
Trigger	Fires automatically on INSERT, UPDATE, or DELETE (BEFORE or AFTER)	Validation, audit logging, cascaded updates

Keep database logic thin: complex business logic is easier to test in application code.



Advanced Queries

Module 11

CTEs (Common Table Expressions)

- Named temporary result set defined with a WITH clause and referenced in the main query
- Improves readability by breaking complex queries into named logical steps
- Multiple CTEs can be chained; each CTE can reference prior ones
- Preferred over deeply nested subqueries for maintainability

Recursive CTEs

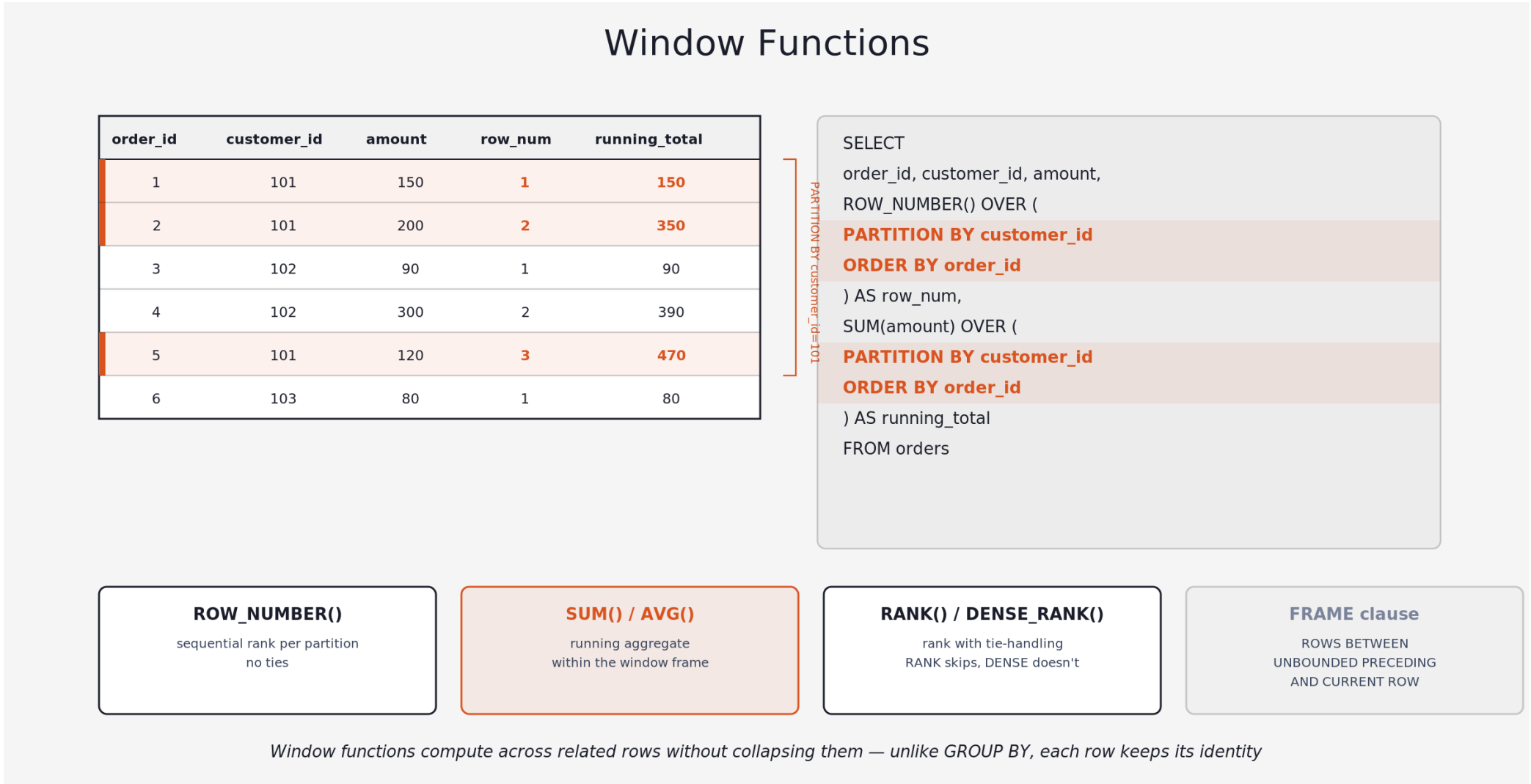
- Recursive CTE: a CTE that references itself to traverse hierarchical or graph-like data
- Structure: anchor member (base case) UNION ALL recursive member (step)
- Use cases: org charts, bill of materials, network paths, folder hierarchies
- MAXRECURSION option limits recursion depth to prevent infinite loops
- Test with small datasets first; deep hierarchies can be expensive

Subqueries

Type	Description	Used in
Scalar	Returns a single value	SELECT, WHERE, HAVING
Row	Returns one row	Comparison operators
Table (derived table)	Returns a result set; requires an alias	FROM clause
Correlated	References the outer query; runs once per outer row, so it can be slow	WHERE, SELECT

Prefer CTEs or JOINs over correlated subqueries for performance.

Window Functions



Analytical Functions

Function	Purpose
NTILE(n)	Divides rows into n buckets; useful for percentile analysis
PERCENT_RANK(), CUME_DIST()	Relative rank within a window
FIRST_VALUE(), LAST_VALUE(), NTH_VALUE()	Retrieve specific values in a window
PIVOT / UNPIVOT	Reshape data from rows to columns and back
CASE WHEN	Conditional aggregation (e.g., count of orders by status)



Query Performance

Module 12

Indexes and EXPLAIN

- Index: an on-disk structure (commonly a B-tree) that lets the engine find rows without scanning the whole table
- Composite indexes: order columns by selectivity and by how queries actually filter/sort
- Every index speeds up reads but slows down INSERT / UPDATE / DELETE, and costs storage — indexing everything is not free
- EXPLAIN / EXPLAIN ANALYZE shows whether the planner actually used an index — required evidence for Assignment 1, Part 6

	Without Index	With Index
Plan	Full table scan	Index range scan
Rows examined (illustrative)	~120,000	~40

Illustrative pattern only — capture your own EXPLAIN ANALYZE numbers for Assignment 1, Part 6.

Query Optimization Checklist

Technique	Why it helps
Use EXPLAIN / EXPLAIN ANALYZE	Shows the execution plan and the bottlenecks
Prefer index seeks over full-table scans	An index B-tree jumps to matching rows; orders of magnitude faster on large tables
Write SARGable predicates	Use bare columns in WHERE; WHERE YEAR(order_date) = 2024 prevents index use, so rewrite it as a date range
Filter early	Push filters down and reduce row counts before joins
Select only needed columns	Avoid SELECT *; reduces I/O
Use partition pruning	Filter on the partition key so the engine skips irrelevant partitions
Keep statistics current	The optimizer uses row counts and distributions to choose plans

Partitioning

- Partitioning enables tables and indexes or index-organized tables to be subdivided into smaller manageable pieces and these each small piece is called a "partition".



oracle-base.com

Advantages of Partitioning

- Smaller and more manageable pieces of data (Partitions)
- Reduced recovery time
- Failure impact is less
- import / export can be done at the " Partition Level".
- Faster access of data
- Partitions work independent of the other partitions.
- Very easy to use



Specialized SQL Features

Module 13

MERGE and UPSERT

Clause	Effect
WHEN MATCHED THEN UPDATE	Updates existing rows that match the condition
WHEN NOT MATCHED THEN INSERT	Inserts rows that do not exist in the target
WHEN NOT MATCHED BY SOURCE THEN DELETE	Deletes rows that are absent from the source
PostgreSQL	Uses INSERT ... ON CONFLICT DO UPDATE (upsert) instead of MERGE

MERGE combines INSERT, UPDATE, and DELETE in a single atomic statement.

JSON in SQL

Engine	JSON support
PostgreSQL	JSON and JSONB types; JSONB is binary-stored, indexable with GIN, and faster
MySQL 5.7+	JSON type with path expressions (->>, JSON_EXTRACT)
BigQuery	JSON type; JSON_VALUE and JSON_QUERY for path extraction

Use JSON for schema-flexible attributes; don't store structured data as JSON blobs.

Full-Text Search in SQL

Engine	Approach
MySQL	Full-text index; MATCH(col) AGAINST('term' IN NATURAL LANGUAGE MODE)
PostgreSQL	tsvector + tsquery; GIN index; supports stemming and stopwords
Elasticsearch / OpenSearch	Advanced search on top of the database

LIKE '%keyword%' is slow on large tables: full table scan, no index. A full-text index is a pre-built inverted index.

PostGIS and Geospatial SQL

- PostGIS: PostgreSQL extension for geospatial data types and queries
- Geometry types: Point, LineString, Polygon; stored as WKT or WKB
- Spatial functions: ST_Distance, ST_Contains, ST_Intersects, ST_Within
- Spatial index: GIST index on geometry columns; accelerates proximity queries
- Use case: find all customers within 5 km of a store; route density analysis



SQL in Python

Module 14

Exercise: Connect to the Database from Python and R




```
#list all columns from table
pd.DataFrame(inspector.get_columns('customers'))
```

	name	type	default	comment	nullable	autoincrement
0	customerNumber	INTEGER(11)	None	None	False	False
1	customerName	VARCHAR(50)	None	None	False	NaN
2	contactLastName	VARCHAR(50)	None	None	False	NaN
3	contactFirstName	VARCHAR(50)	None	None	False	NaN
4	phone	VARCHAR(50)	None	None	False	NaN
5	addressLine1	VARCHAR(50)	None	None	False	NaN
6	addressLine2	VARCHAR(50)	None	None	True	NaN
7	city	VARCHAR(50)	None	None	False	NaN
8	state	VARCHAR(50)	None	None	True	NaN
9	postalCode	VARCHAR(15)	None	None	True	NaN
10	country	VARCHAR(50)	None	None	False	NaN
11	salesRepEmployeeNumber	INTEGER(11)	None	None	True	False
12	creditLimit	DECIMAL(10, 2)	None	None	True	NaN

SQL in Python Pipelines

- SQLAlchemy: ORM + Core for database-agnostic SQL from Python
- psycopg2 / pg8000: PostgreSQL drivers; use connection pooling in production
- dbt: SQL-first transformation framework with testing and lineage tracking
- Parameterized queries prevent SQL injection: never concatenate user input into SQL
- Use COPY / LOAD DATA / BigQuery Storage Write API for bulk inserts, not row-by-row
- Course labs: PythonMySQL.ipynb (pymysql basics), Python-Pipeline.py (ETL), AISQLAgent.ipynb (LLM-generated SQL)

Connecting Python to MySQL with pymysql

- pymysql: pure-Python MySQL client; drop-in replacement for MySQLdb
- Always use parameterized queries (%s placeholders); never f-string SQL
- Wrap DML in try/except with conn.rollback(); call conn.commit() on success

```
conn = pymysql.connect(host=…, user=…, password=…, database='classicmodels')
cursor = conn.cursor(pymysql.cursors.DictCursor)
cursor.execute('SELECT * FROM customers WHERE country = %s', ('USA',))
rows = cursor.fetchall()   # pd.DataFrame(rows)
```

AI SQL Agent: Natural Language to SQL

- How an AI SQL agent works:
  1. Give the LLM the database schema (CREATE TABLE statements) as context
  2. A user asks a question in natural language, e.g. "Which product line generated the most revenue last year?"
  3. The LLM writes the SQL
  4. Validate the SQL, then run it on a read-only connection
  5. Review the result: check joins and column names
- Lab: AISQLAgent.ipynb (LangChain + OpenAI GPT against the ClassicModels schema)
- Limitations: hallucinated column names and incorrect joins; always verify the output

Resources & Further Reading

- Docs and guides:
  - MySQL Reference Manual: dev.mysql.com/doc
  - Use The Index, Luke: SQL indexing and performance (use-the-index-luke.com)
  - ClassicModels sample database: the MySQL tutorial dataset used throughout this course
- Course labs:
  - SQL-3.1-Intro.sql through SQL-3.3-GroupBy.sql: progressive exercises on ClassicModels data
  - SQL-3.7-Views.sql and SQL-3.8-DDL.sql: views and DDL labs
- Books:
  - SQL Performance Explained, Markus Winand
  - Designing Data-Intensive Applications, Ch. 3 (Storage and Retrieval), Kleppmann

Summary

- SELECT, WHERE, GROUP BY, HAVING, and JOINs are the core of analytical SQL.
- Nested queries, views, and CTEs break complex logic into readable, reusable pieces.
- DDL defines structure, DML reads and changes data, TCL and DCL control transactions and access.
- Window and analytical functions answer ranking and running-total questions without collapsing rows.
- Indexes, EXPLAIN, and SARGable predicates make queries fast.
- Wrap multi-statement writes in transactions so failures roll back cleanly; use parameterized queries from Python.